LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

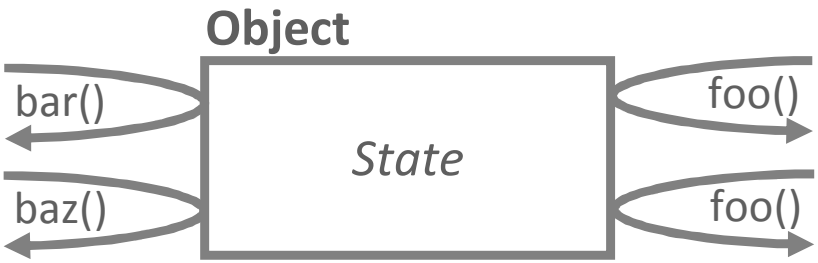
Concurrent Objects

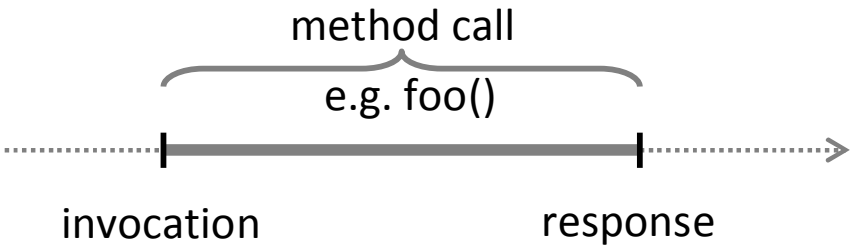


Overview

- Objects
  - Sequential specification
  - Concurrent objects
- Safety properties – what do we consider correct behavior?
  - Sequential consistency
  - Quiescent consistency
  - Linearizability
- Progress properties – how can threads influence the progress of executions?
  - Blocking vs. Non-Blocking
  - Wait-free vs. Lock-free

Objects

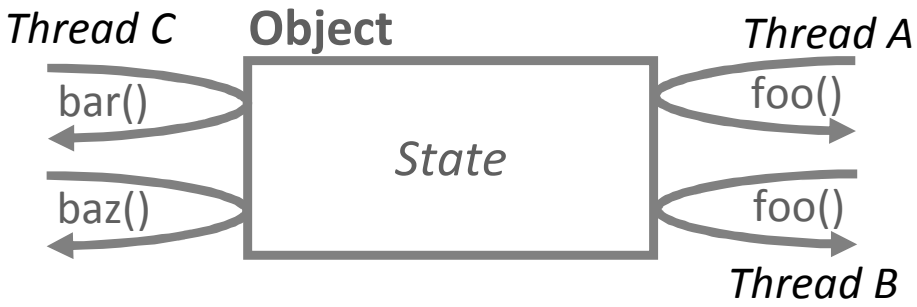


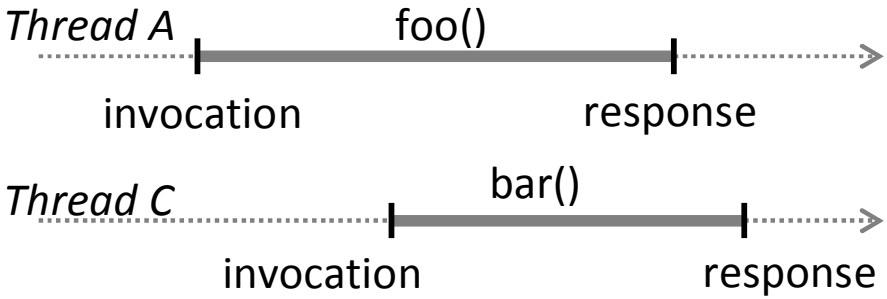


- Objects in general
  - Encapsulate data (the object's **state**) and behaviour (**member functions** / **methods**)
  - **Methods** are the only way to change the state of an object

- **Sequential specification**: for each method we need to specify
  - **Precondition**: the state the object needs to be in for the method call
  - **Postcondition**: the state the object is in after the method call (plus any return values of the call)
  - For sequential execution, we don't need to worry about intermediate states while the call is in progress

Concurrent Objects





- Concurrent objects
  - Can be used concurrently by multiple threads
  - I.e., concurrent method invocations are allowed

- Questions to consider
  - Can method calls see the **incomplete effects** of other, concurrent method calls?
  - Do we have to specify the effects of all possible concurrent interactions between method calls in a **concurrent specification**?

Correctness and Progress

- **Two aspects** that we need to consider, when designing and using concurrent objects:
  - What do we consider to be the **correct results** of concurrent executions? These are so-called **safety** properties.
  - How do we describe and manage the (in)abiliy of some threads to **make progress** with their method invocations in the presence of other threads (and their method invocations)? These are so-called **liveness** properties.

- I.e., threads may interfere with each other both in terms of correctness and in terms of progress
  - **Correctness**: The effects and results of method calls
  - **Progress**: The ability of threads to complete their method calls

Progress Example - Blocking

```
class Counter {
  Lock lock;
  int ctr=0;

int get_next_value():
  lock.acquire();
  int c = ctr++;
  lock.release();
  return c;
}
```

- Lock-based concurrent object
  - This is called a **blocking** implementation
  - **Blocking** means that the delay of a thread can delay all other threads
  - **Non-blocking**: the delay of a thread cannot delay all other threads
- Reasons for delays:
  - Cache or page misses, OS scheduler preemption
  - Extreme cases: a thread might be delayed forever – e.g., killed or deadlocked
- This may or may not be a problem in practice
  - E.g., real-time embedded systems

Correctness – Sequential Consistency as a Guideline

- SC for memory systems:
  - The memory system appears to serve one request after the other (**atomically** and in **isolation**)
  - This results in a total global order of all memory accesses
  - Each thread's memory requests appear in its **program order** in this total global order
  - We can apply the same idea to concurrent objects:

- **Def: Sequential Consistency** for concurrent objects
  - Method calls **appear to be executed one-at-a-time** (atomically and in isolation)
  - Method calls **appear to take effect in program order**

- **Discussion**
  - The one-at-a-time property defines a total global order of method calls on the object
  - Each thread's requests appear in program order in this total global order

Load-Store Registers

- Example: Object form of a single memory location – called a **register**

```
class Register {
  int val=0;

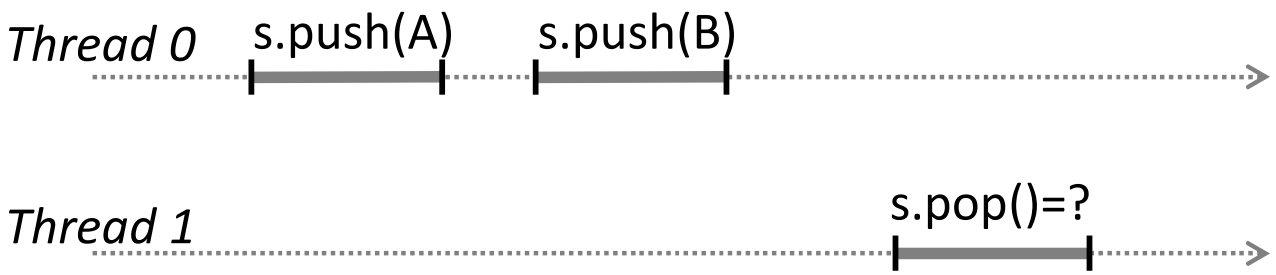
load():
  return val;

store(int v):
  val=v;
}
```

- If Register offers SC, then
  - A load() will always return the value of a previous store() or the initial value. I.e., the bit-patterns of two concurrent stores will not get mixed up
- If Register is used by just a single thread, it will behave like a regular sequential object
  - So for example, if the thread issues two store() operations followed by a load(), then the load will return the value of the most recent store()

Problems with SC for Objects (1)

- As stated, the SC condition doesn't take the **real-time behaviour** into account at all



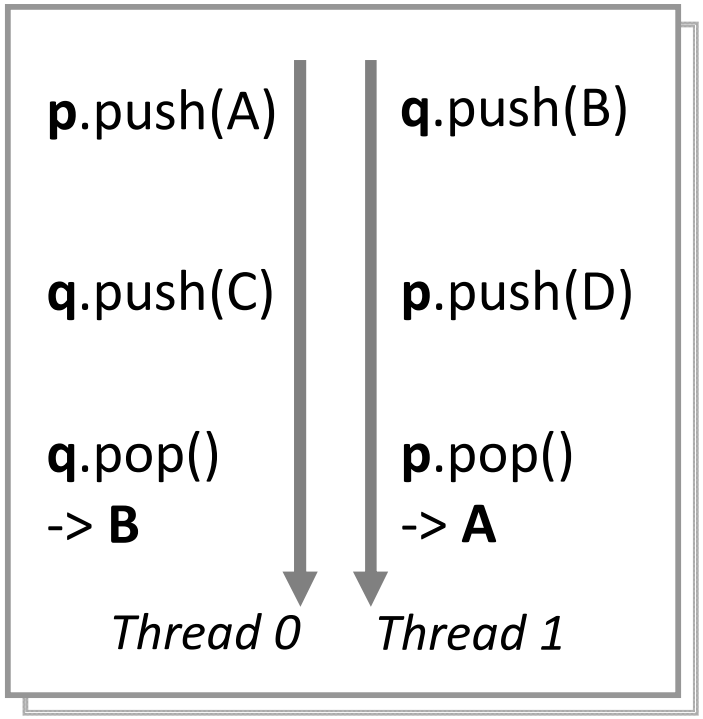
- Example stack s:
  - Thread 0 does push(A), push(B)
  - Thread 1 does pop() after thread 0 (in real time)
  - Then pop() returning "empty" is perfectly fine for SC
  - As is pop() returning the result of the last push (B) or the first push (A)

Problems with SC for Objects (2)

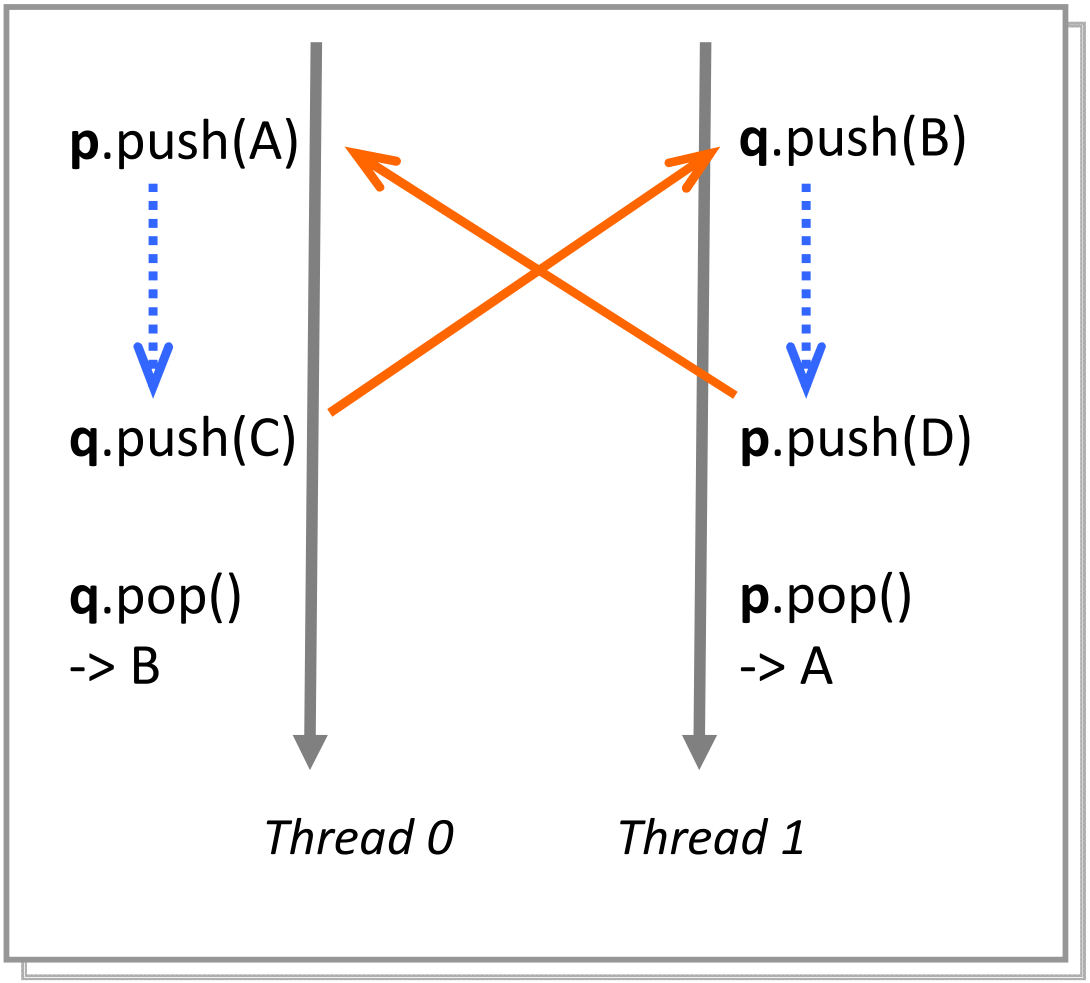
- SC is not **compositional**
  - If we are composing a system of SC objects, that doesn't mean that the whole system is SC
  - So we can't verify/test the whole system by testing the components
- The lack of composability is not generally a problem for memory systems
  - Essentially the memory system is a giant monolithic concurrent object, not part of a larger system

Composition of SC is not SC – Example (1)

- Assume two SC stacks **p** and **q**, used by two threads, 0 and 1
  - Stack is LIFO – last in, first out
- Discussion
  - Since **p** is sequentially consistent, **p**.pop() may result in A or D, let's say the result is **A**
  - Since **q** is sequentially consistent, **q**.pop() may result in B or C, let's say the result is **B**
- **Q**: Can we find a sequentially consistent order of all operations in the system?
  - I.e., an execution in which the program order in each thread is obeyed?
- **A**: No! No such sequentially consistent ordering of all operations can be found



Composition of SC is not SC – Example (2)



Ordering imposed by the semantics of a stack

Ordering Imposed by program order

- Taken together we have an ordering loop
  - Thus no sequentially consistent ordering can be found

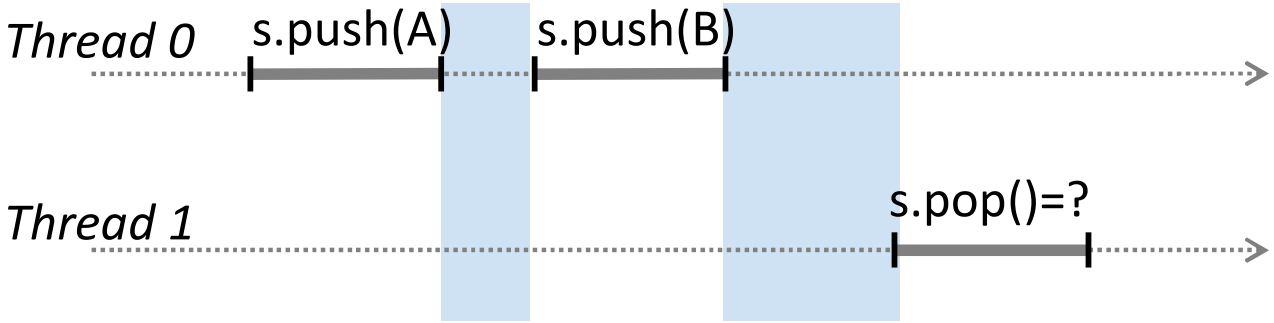
Alternatives to SC: Quiescent Consistency

- Quiescent consistency does take real-time behavior into account
  - **Quiescent phase** for an object: a duration of time when no method calls on the object are in progress

- **Def: Quiescent Consistency**
  - Method calls **appear to be executed one-at-a-time** (atomically and in isolation), like with SC
  - Method calls **separated by quiescent phases appear to take effect in real-time order**

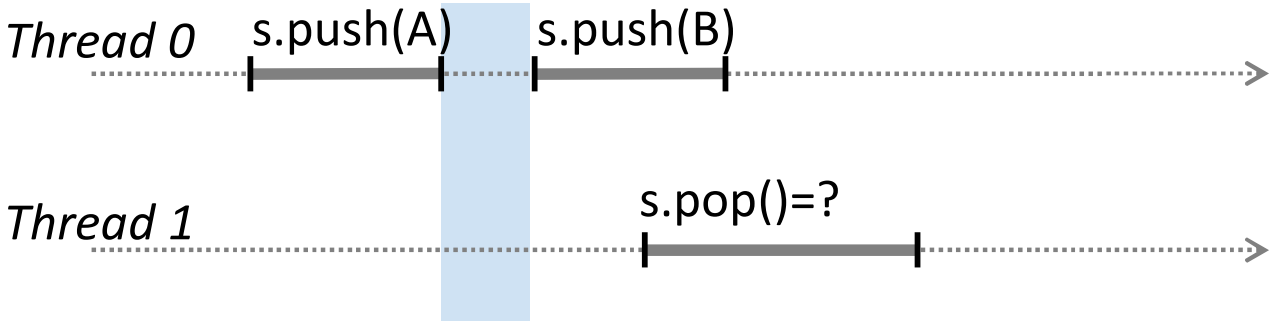
- Discussion
  - Phases of non-quiescence alternate with quiescent phases
  - In quiescent phases a "stable state" is established that corresponds to **some arbitrary interleaving** of the method calls in the previous non-quiescent phase

Quiescent Consistency Example



Quiescent Phases

- s.pop() must return B



- s.pop() may return A or B

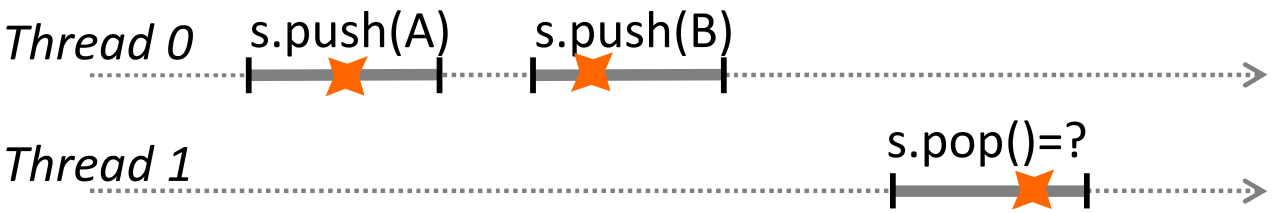
Alternatives to SC: Linearizability

- Linearizability is a strictly stronger property than SC
  - Every implementation that is linearizable is also SC
  - Linearizability is compositional
  - Linearizability takes the real-time behavior into account

- **Def: Linearizability**
  - Method calls **appear to be executed one-at-a-time** (atomically and in isolation), like with SC
  - Method calls **appear to take effect instantaneously at some point between method invocation and response (the "linearization point")**

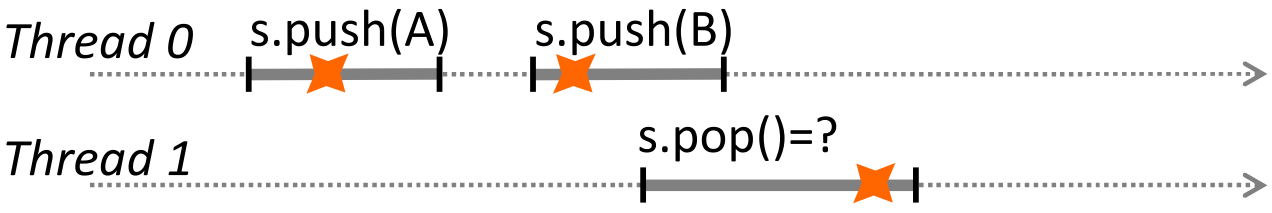
- Linearization points
  - For lock-based implementations anywhere in the critical section will work as the linearization point
  - For lock-free implementations linearization points are typically the points where we atomically swing a pointer or similar operations

Linearizability Example

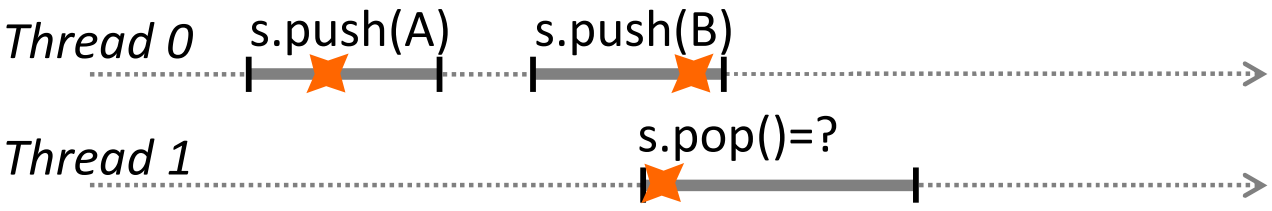


s.pop() returns B

Linearization Point



s.pop() returns B



s.pop() returns A

Correctness Summary

- We have seen several formalizations for what we consider to be "correct" concurrent behavior of a concurrent object
  - **Linearizability** is the most widely used specification and also what we will be using in the rest of the course
  - All notions are based on equivalence with sequential execution of some interleaving of the methods
  - This makes live a lot easier, no need to consider partial effects of method calls in a concurrent specification

- Other formulations exist too, e.g., Serializability

Progress Conditions

- **Blocking**
  - The delay of one thread **can delay all others**

- **Starvation-free**
  - **Every method** call finishes in a finite number of steps(*)

- **Deadlock-free**
  - **Infinitely often some method** calls finish in a finite number of steps(*)

(*): If, and only if, the underlying system provides us with some guarantees, such as that every thread eventually leaves its critical section.

Dependent Progress Conditions

- **Non-blocking**
  - The delay of one thread **cannot delay all others**

- **Wait-free**
  - **Every method** call finishes in a finite number of steps (**no thread can be delayed**)

- **Lock-free**
  - **Infinitely often some method** calls finish in a finite number of steps (**not all threads can be delayed**)

Independent Progress Conditions

- **Obstruction-free**
  - If a method **executes in isolation** (no other threads take steps), it **finishes in a finite number of steps**

Progress Discussion (1)

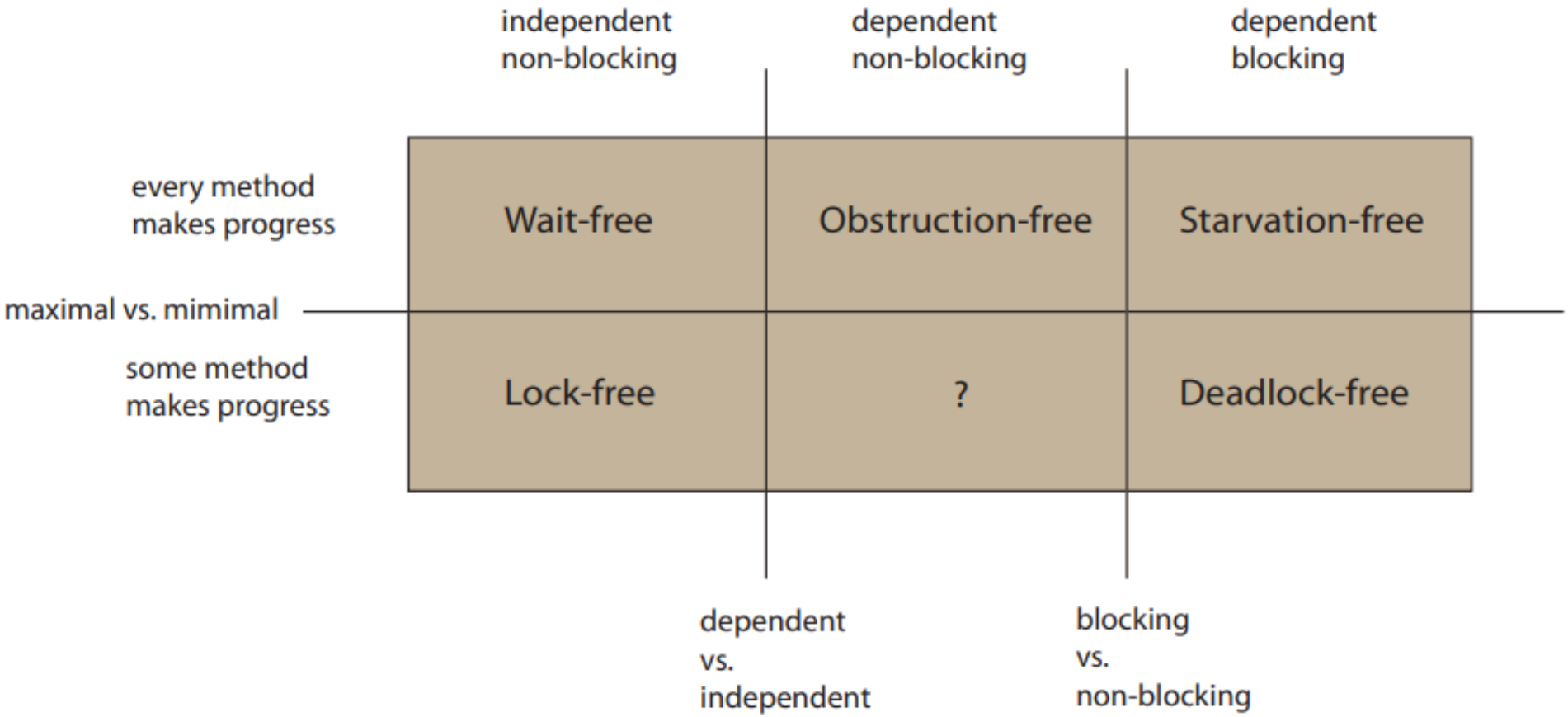
- The progress conditions can apply to **individual methods** or **whole objects**
  - I.e., An object x is wait-free if every method is wait-free, e.g. a wait-free stack which offers a wait-free push() and wait-free pop() methods
- All definitions are in terms of “steps”
  - This abstractly refers to the computation that has to be done in each method independent of an actual implementation (where each step could take different amounts of time)
- Wait-free implies Lock-free
  - But not the other way around
- Starvation-free implies Deadlock-free
  - But not the other way around

Bit more formal definitions…

A method foo() is…				
wait-free	if the following holds:	If there is a thread T with a pending invocation of foo()	and thread T keeps taking steps	then the invocation of foo() by thread T completes in a finite number of steps.
lock-free			and thread T keeps taking steps	then some pending invocation of foo() by some thread completes in a finite number of steps.
starvation-free	if the following holds:	If there is a thread T with a pending invocation of foo()	and every thread with a pending invocation of foo() keeps taking steps	then the invocation of foo() by thread T completes in a finite number of steps.
deadlock-free			and every thread with a pending invocation of foo() keeps taking steps	then some pending invocation of foo() by some thread completes in a finite number of steps.

- Independent progress condition means that we don't require cooperation from other threads (or a fair scheduler)
- Maximal progress means every method makes progress
- Minimal progress means some methods make progress

Image source: Herlihy, M., & Shavit, N. (2011, December). *On the nature of progress.* In International Conference On Principles Of Distributed Systems (pp. 313-328). Springer, Berlin, Heidelberg.



Progress Discussion (2)

- In the rest of the lecture we will focus on concurrent objects in three flavors
  - **Blocking** concurrent objects (lock-based implementations)
  - **Lock-free** concurrent objects
  - **Wait-free** concurrent objects

- There are different trade-offs to be considered between these options
  - In terms of **performance** lock-free or wait-free **may** offer benefits
  - In terms of **reliability** lock-free and especially wait-free approaches have inherent advantages
  - In terms of **implementation complexity**, lock-free and wait-free approaches are much more challenging; the potential for errors is much higher